

HEXA

Governance

The control and trust layer. Holds identity, tenant isolation, permissions, approvals and the tamper-evident record, so every mission stays inside the authority of the person who started it.

Who this is for: product, engineering, operations and anyone who has to explain XELOR to somebody else.

What it covers: the work HEXA reasons about, the rules it cannot break, the exact tools it can call, and what is honestly not built yet.

Truth standard: the repository as it stands on 1 August 2026.

HEXA

What HEXA is for

A role with a fixed tool list, not a general assistant

In one sentence

Holds identity, tenant isolation, permissions, approvals and the tamper-evident record, so every mission stays inside the authority of the person who started it.

3

business areas it speaks for

2

tools it can actually call

4

capability families allowed

0

agents it may delegate to

What reaches it

A verified token · The requested capability and permission · The proposed action plan · The recorded human decision

What it hands back

Allow or refuse, with the evidence · Preflight and post-execution verification · An approval requirement · A hash-chained, append-only trail

Capability families it is allowed to touch

agent.

workflow.

governance.

general.

Ownership and authority are not the same thing

The areas above are where HEXA is the right specialist to ask. The tool table on page 6 is the much smaller set it can invoke at runtime. Owning a business area does not grant runtime power over it — and for ONYX, the supervisor, the gap between the two is the widest in the system.

How HEXA's world actually runs

The business pipeline, not the agent plumbing

Four independent gates stand between a request and a row. Each is enforced by machinery rather than by wording, and three of the four are checked by CI on every build.

1

Identity

The tenant comes from a signature-verified Keycloak token's groups claim — never a request header. A proxy header is exactly the attack CVE-2025-29927 describes.

2

Tenant fence

withTenant() opens the transaction and sets app.current_tenant transaction-locally. Every tenant table has FORCE RLS; an unset tenant resolves to NULL and matches nothing.

3

Permission

138 keys shaped module.entity.action, typed at compile time. A permission that is not in the catalogue cannot even be granted — a trigger refuses the insert.

4

Approval

W1 resolves the approver from data — a named user or a role — and refuses anyone else by name with 403. The definition version is pinned, so publishing v2 never rewrites an approval in flight.

5

Record

Each audit row is sha256(previous hash + the canonicalised entry), sequenced per tenant. The append-only trigger fires for the schema owner too.

How much of this HEXA can actually see

Of everything above, HEXA can read exactly one thing directly — company masters. The rest of this pipeline is context for judging what it reads; it is not data the agent can reach. Where a mission needs more, it asks the specialist who owns it or it says the evidence is missing.

What the code will not let anyone do

Enforced by constraints and locks, not by wording

app_user owns nothing and cannot bypass RLS

The API connects as a NOBYPASSRLS role that owns no table, so FORCE RLS is a real backstop rather than decoration.

The chain names how it broke

Verification distinguishes a deleted row (sequence gap), a replaced row (link mismatch) and an edited row (hash mismatch) — and stores the verdict, because “we verify nightly” is a claim and a dated row is evidence.

Ledger-critical writes never ride the bus

Stock, journals and budget commit synchronously in the caller's transaction. The outbox carries side effects only.

CI enforces the fence

db:rls-check asserts FORCE RLS and that a policy really keys on app.current_tenant; a two-tenant leak probe proves it live; perm-check reconciles all 138 routes against the registry.

Why this page matters more than the agent pages

An agent is only as trustworthy as the system it reads. Every rule here holds whether the request came from a person, a screen, a seeder or HEXA — which is precisely why an agent can be given a read tool without also being given a way to do damage.

Where these rules are kept

apps/api/src/common/tenant.middleware.ts

packages/platform/src/audit/hash-chain.ts

apps/api/src/modules/workflow/w1.service.ts

How HEXA takes part in a mission

The engine controls order, parallelism and recovery



1 · Scope

ONYX turns the goal into a run against a frozen graph version, with a fixed tenant, user, step budget and timeout.

2 · Authorise

Two checks, both required: the tool must be on HEXA's allow-list, and the signed-in person must independently hold the permission.

3 · Execute

A registered domain service does the work under the same rules a screen would face. There is no general SQL doorway.

4 · Preserve

Inputs, outputs, events and a checkpoint after every wave, so the run can be explained or resumed.

An agent can narrow authority, never widen it

HEXA is a specialist and delegates to nobody. Because the permission check is repeated against the requesting person, a mission can never reach data that person could not have opened themselves.

Everything HEXA can call

This table is the complete list — there is no other door

Capability	Mode	What comes back	Permission required	Needs approval
general.companies.read	Read	Company masters	general.company.read	No
agent.action.dispatch	Execute	Governed work item	agentos.run.operate	Yes

Read · Analyse · Simulate

Return evidence or a reproducible view. Nothing in the business changes.

Draft

Produce reviewable content that is labelled a draft and can never present itself as an approved outcome.

Execute

The only side-effecting mode in the whole system, and the only one that requires an approved human gate above it.

Both locks must open

The agent's allow-list AND the person's permission. Either one failing is a refusal.

What “dispatch” does and does not do

It appends a governed work item naming the responsible specialist, the approval it came from and the exact payload. It does not change a sales order, a stock balance, a production order, a journal or a customer message — a person still does that work.

Where HEXA actually appears

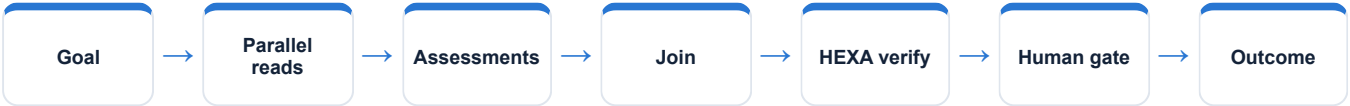
Node by node, from the registered graph definitions

Mission graph	What HEXA does in it
Cross-functional readiness	Verifies policy and evidence before the human gate
Seven-agent operating review	Reads company context, assesses control, runs four checks
Controlled action mission	Context, assessment, preflight, one dispatch, outcome verification — 5 nodes
Working Capital Review	Verifies every figure ties to source and no posting occurred
QMS & Audit Readiness	Verifies traceability and that no AI decided a result

Reading this honestly

“Not involved” is not a limitation, it is the design. A mission asks only the specialists whose evidence it needs, and a specialist cannot add itself to a graph at runtime. The graph is written down, versioned and content-hashed before the run starts.

The shape every mission shares



Can this persona open another department?

Real records from the running demo, not an illustration

Signed in as mica.commercial, five of the six departments are visibly shut — shown and dimmed, not hidden, because pretending they do not exist would redraw the company.

Typing /department/SPAR into the address bar does not help. The SERVER refuses: the same token asking the API for another department's data gets 403, resolved from grants in tenant-fenced tables.

Nothing in between could be edited in a browser's developer tools, because nothing in between is making the decision.

The sentence to say out loud

“HEXA contributes evidence from its own area, with the source records behind it and its limits stated. It does not claim an action is done until the system has recorded and verified it.”

Fair questions to ask while watching

- Which records is this answer standing on?
- Which permission was checked, and against whom?
- Is this a recorded fact, a simulation, a draft, or a completed result?
- Would this step have waited for a person?
- What happens if the service restarts halfway through?

Live today, and deliberately not

The second column is the one worth reading

Working now • FORCE RLS on every tenant table, CI-verified • 138 typed permissions with a catalogue trigger • W1 approvals with a hash-chained trail • Audit verification, anchoring and the auditor pack • Circuit breakers, DLQ triage and signed webhooks as pure, tested logic	Not built — stated plainly • The permission guard does not filter is_active, so a soft-revoked grant is still honoured — a real divergence from the access simulator • W1 has no cancel, no delegation and no escalation; the SLA is a pull query with no timer • Row-scope and field masking are reachable only through the access preview, not applied by list endpoints • Integration has no transport: the resilience logic is real and tested, the I/O half is simulate-driven
---	--

Identity boundary	Verified token → tenant and actor
Authority boundary	Agent allow-list AND the person's permission
Action boundary	Side effects need an approved ancestor node
Data boundary	Domain service over a tenant-fenced database
Evidence boundary	Append-only runs, events, checkpoints and audit

Gaps are listed because a guide that hides them fails the first hour of technical due diligence. Every item on the right is either a roadmap decision or a known defect, and both are recorded in the project's own capability-gap register.

HEXA on one page

For explaining the agent to somebody new

Its job
Holds identity, tenant isolation, permissions, approvals and the tamper-evident record, so every mission stays inside the authority of the person who started it.

Speaks for
Organisation & master data, Administration, Integration

Takes in
A verified token · The requested capability and permission · The proposed action plan · The recorded human decision

Gives back
Allow or refuse, with the evidence · Preflight and post-execution verification · An approval requirement · A hash-chained, append-only trail

Can call
general.companies.read, agent.action.dispatch

Cannot do
The permission guard does not filter is_active, so a soft-revoked grant is still honoured — a real divergence from the access simulator · W1 has no cancel, no delegation and no escalation; the SLA is a pull query with no timer · Row-scope and field masking are reachable only through the access preview, not applied by list endpoints

Words used on these pages

Agent	A named specialist with a fixed, allow-listed set of tools — not a process that can roam.
Capability	One registered operation with a declared mode, owner and required permission.
Mission graph	A versioned recipe: which steps run, which run together, where it waits for a person.
Checkpoint	A stored snapshot after each wave, used to explain a run and to resume it safely.
Governed work item	An approved, append-only assignment for a human team — not the business change itself.